Gradient Boosting

What is Gradient Boosting?

An ensemble learning algorithm that builds models sequentially. Each new model tries to correct the errors of the previous models by minimizing a loss function using gradient-based optimization.

Ensemble
Many simple models work together

Sequential
Each model learns from the last

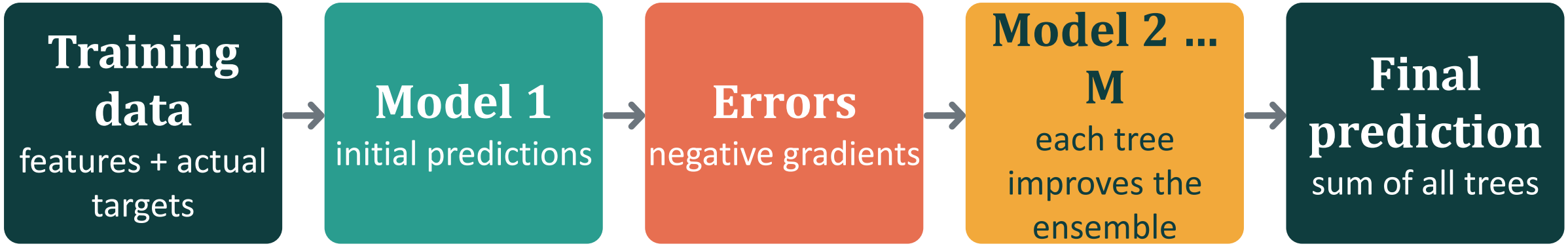
Gradient-based
Minimizes a loss function

Final prediction = combined contributions of all models

Gradient Boosting Algorithm Steps

1. Build the first model
2. Find where its predictions are wrong
3. Train the next model to correct those errors
4. Repeat this process
5. Combine the predictions of all models into the final result

Overall Flow of the Algorithm



↺ Errors → next tree → update → repeat for M rounds

$$F_m(x) = F_{m-1}(x) + \eta h_m(x)$$

2. Why Do We Need Gradient Boosting?

Student	Study hours	Actual marks
A	1	40
B	2	50
C	3	60
D	4	70
E	5	80

Goal: predict a student's marks from study hours

Using the Errors

- Gradient Boosting uses these errors to guide the next model
- For squared error, the next tree learns the residuals (actual − prediction)
- The next tree does not start from scratch
- It learns how to improve the existing ensemble

Negative residual → overpredicted · Positive residual → underpredicted

3. What is a Weak Learner?

Moderately accurate alone, but useful when combined. Usually a shallow regression tree (a stump has one split).

1

Tree 1

Predicts the general level of marks.

2

Tree 2

Corrects students with fewer study hours downward.

3

Tree 3

Corrects students with more study hours upward.

Each tree is simple — the ensemble becomes accurate

4.1 The Prediction Equation

$F_0(x)$ initial prediction

$h_m(x)$ correction learned by the m-th tree

M total number of trees

η learning rate: how much each tree contributes

$$F(x) = F_0(x) + \Sigma\, \eta h_m(x), \quad m = 1 \ldots M$$

Negative Gradient for Squared Error

Loss

$$L_i = \tfrac{1}{2}(y_i - F(x_i))^2$$

→

Derivative

$$\partial L_i / \partial F(x_i) = F(x_i) - y_i$$

→

Negative gradient

$$r_i = y_i - F(x_i)$$

This is exactly the residual: actual value minus current prediction.

Negative gradient = $y - F(x)$

Negative Gradients: Example

Actual	Current prediction	Negative gradient
40	60	−20
50	60	−10
60	60	0
70	60	+10
80	60	+20

For squared error, negative gradient = residual

5. The Algorithm Step by Step

1

Initialize: F_0 = mean of the targets

2

Calculate residuals / negative gradients

3

Train a new decision tree on the residuals

4

Update: $F_m = F_{m-1} + \eta h_m(x)$

5

Repeat for the required boosting rounds

6

Predict with the accumulated model

Initialize → Residuals → Tree → Update → Repeat → Predict

Step 1: Initialization

$$F_0(x) = (1/N) \Sigma y_i$$

For squared-error regression, the best constant starting prediction is the mean of the target values.

(40+50+60+70+80) / 5

= 60

Every student starts with the same prediction: $F_0 = 60$

6. Numerical Example: Setup

Study hours (x)	Actual marks (y)
1	40
2	50
3	60
4	70
5	80

Rules for this example

- Learning rate $\eta = 0.5$
- Fixed splits: Hours ≤ 3 | Hours > 3
- Each leaf predicts the average residual
- Three boosting rounds

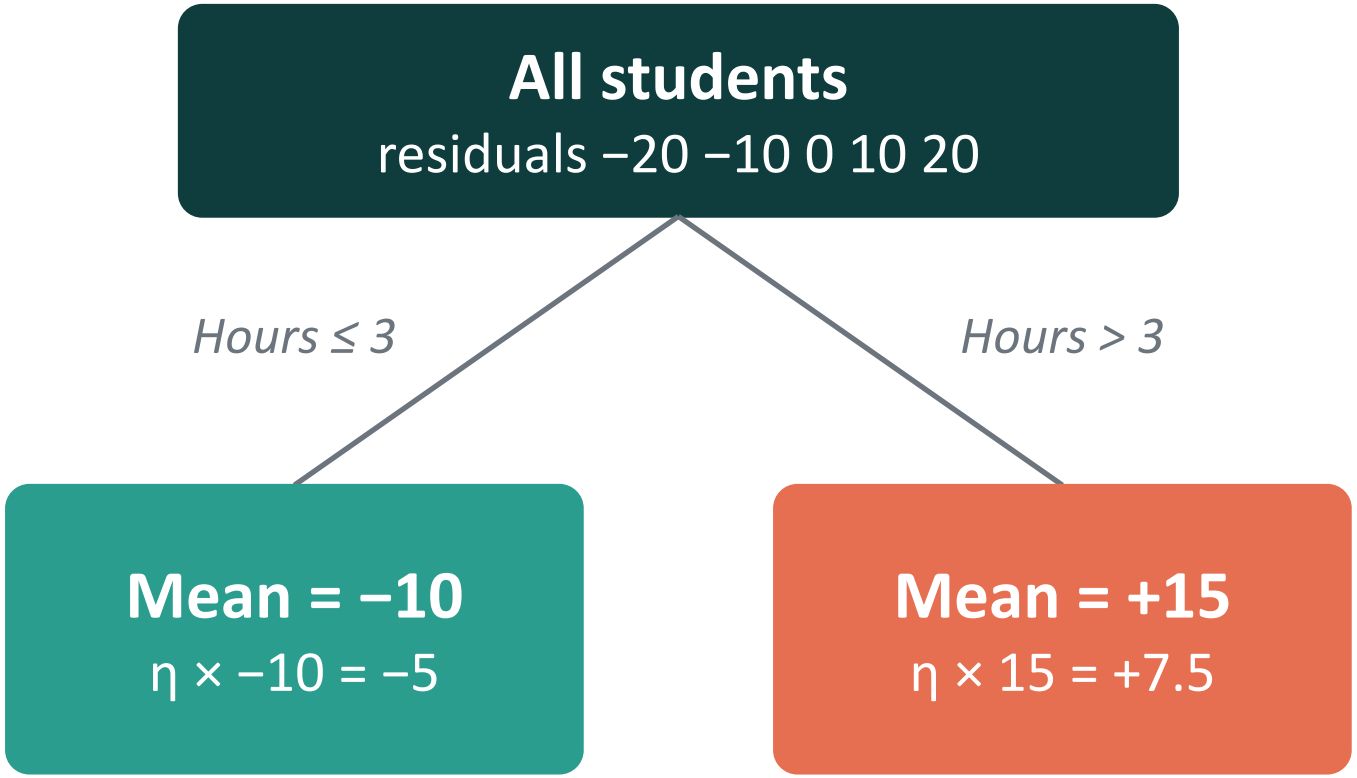
Round 0 → Tree 1 → Tree 2 → Tree 3

Round 0: Initial Prediction

Hours	Actual	Prediction	Residual
1	40	60	−20
2	50	60	−10
3	60	60	0
4	70	60	+10
5	80	60	+20

$$F_0 = (40 + 50 + 60 + 70 + 80) / 5 = 60$$

Round 1: Train Tree 1



Leaf means

Hours ≤ 3: (−20−10+0)/3 = −10

Hours > 3: (10+20)/2 = +15

With η = 0.5 the corrections are −5 and +7.5.

$$F_1(x) = F_0(x) + 0.5\, h_1(x)$$

Round 1: Updated Predictions

Hours	Actual	F_0	Tree 1	F_1
1	40	60	−5	**55**
2	50	60	−5	**55**
3	60	60	−5	**55**
4	70	60	+7.5	**67.5**
5	80	60	+7.5	**67.5**

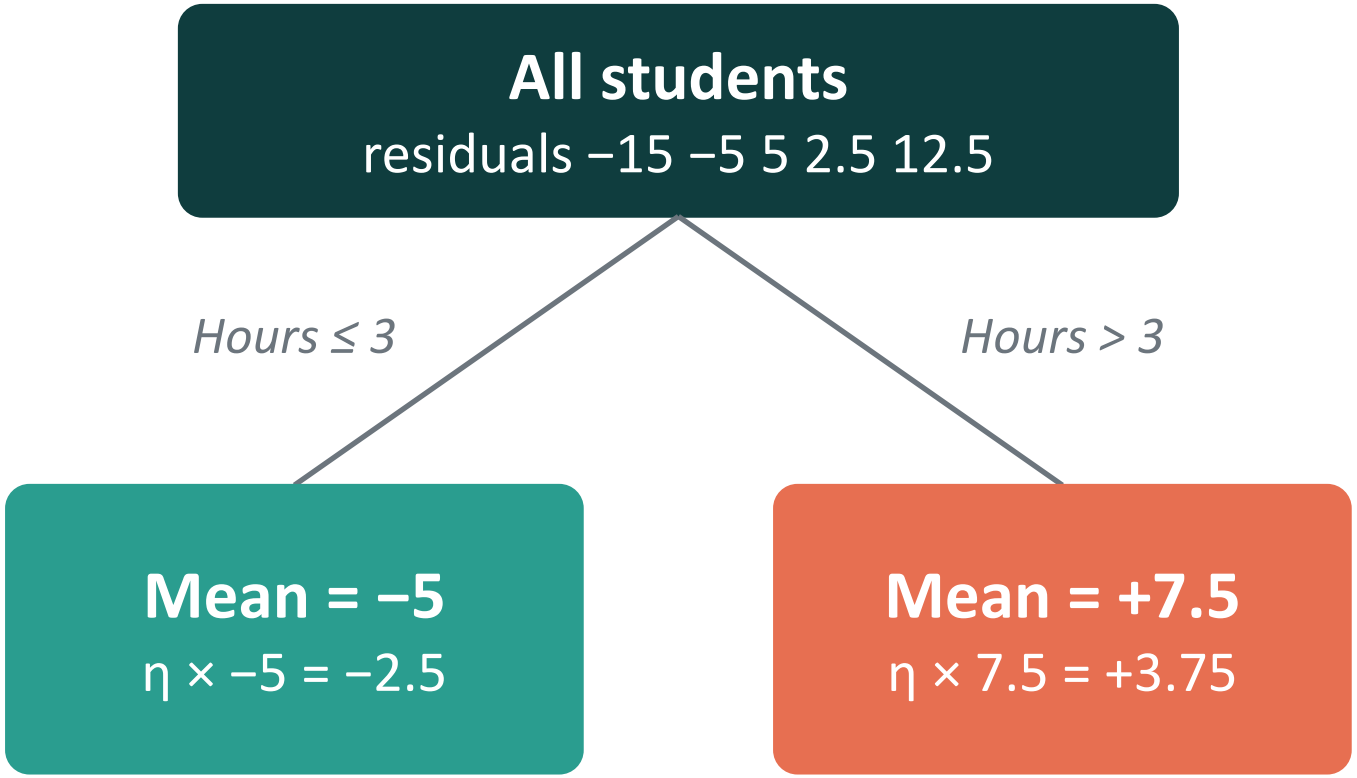
The first tree moves the predictions toward the actual marks

Round 2: New Residuals

Hours	Actual	F_1	Residual
1	40	55	−15
2	50	55	−5
3	60	55	+5
4	70	67.5	+2.5
5	80	67.5	+12.5

Errors are recalculated from the updated predictions

Round 2: Train Tree 2



Leaf means

Hours ≤ 3: (−15−5+5)/3 = −5

Hours > 3: (2.5+12.5)/2 = 7.5

Same two groups for simplicity.

$$F_2(x) = F_1(x) + 0.5\, h_2(x)$$

Round 2: Predictions After Tree 2

Hours	Actual	F_1	Tree 2	F_2
1	40	55	−2.5	**52.5**
2	50	55	−2.5	**52.5**
3	60	55	−2.5	**52.5**
4	70	67.5	+3.75	**71.25**
5	80	67.5	+3.75	**71.25**

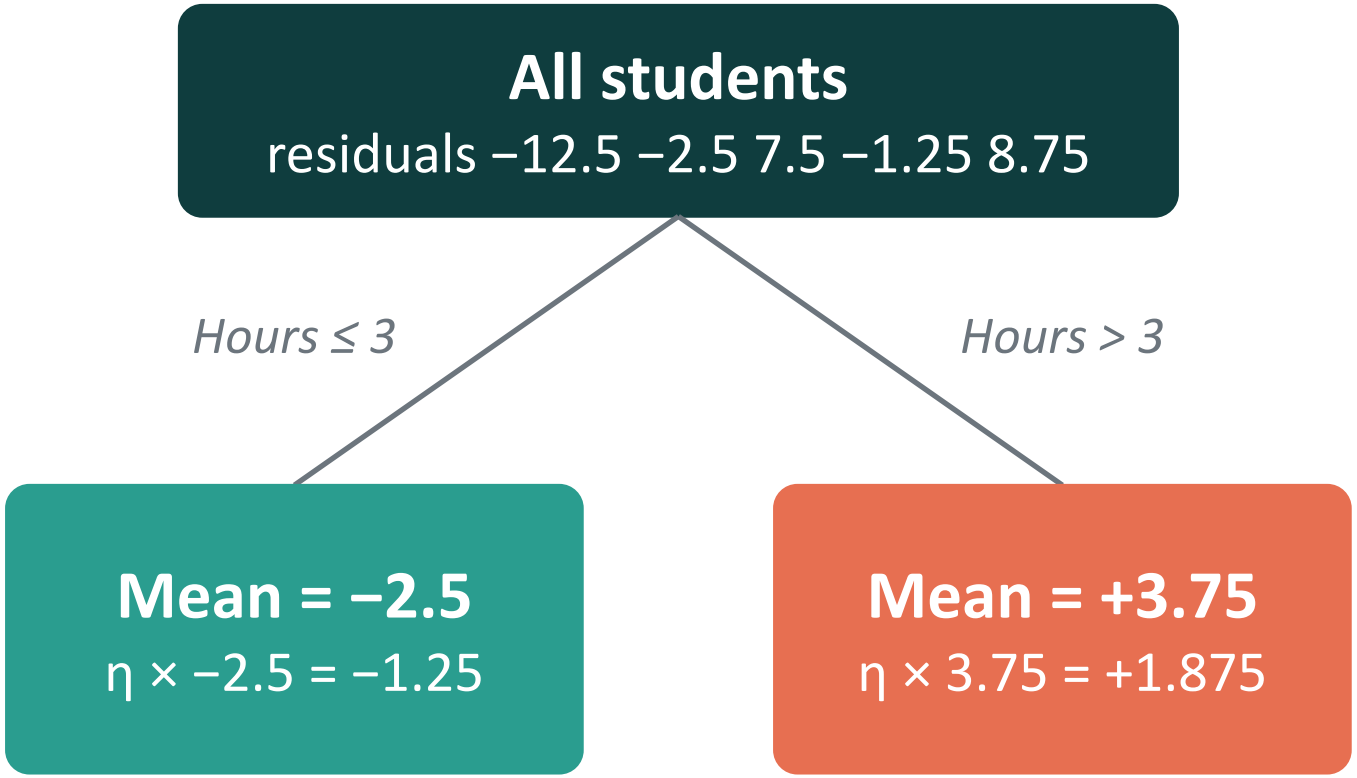
Hours 3 moves away (error 5 → 7.5), yet total loss falls

Round 3: New Residuals

Hours	Actual	F_2	Residual
1	40	52.5	−12.5
2	50	52.5	−2.5
3	60	52.5	+7.5
4	70	71.25	−1.25
5	80	71.25	+8.75

Residuals shrink every round — new errors from the updated model

Round 3: Train Tree 3



Leaf means

Hours ≤ 3: (−12.5−2.5+7.5)/3 = −2.5

Hours > 3: (−1.25+8.75)/2 = 3.75

Corrections: −1.25 and +1.875.

$$F_3(x) = F_2(x) + 0.5\, h_3(x)$$

Round 3: Final Predictions

Hours	Actual	F_2	Tree 3	F_3
1	40	52.5	−1.25	**51.25**
2	50	52.5	−1.25	**51.25**
3	60	52.5	−1.25	**51.25**
4	70	71.25	+1.875	**73.125**
5	80	71.25	+1.875	**73.125**

Prediction after three rounds

Summary of the Three Rounds

Hours	Actual	F_0	F_1	F_2	F_3
1	40	60	55	52.5	51.25
2	50	60	55	52.5	51.25
3	60	60	55	52.5	51.25
4	70	60	67.5	71.25	73.125
5	80	60	67.5	71.25	73.125

Sum of squared errors: $1000 \rightarrow 437.5 \rightarrow 296.9 \rightarrow 261.7$

Fixed-split trees for illustration; a real implementation chooses splits and leaf values from its objective.

Key Lesson from the Example

1

Fix errors

Each new tree learns from the current model's remaining errors.

2

Small steps

Predictions are corrected a little at a time, scaled by η.

3

Loss falls

Total loss decreases round after round.

Prediction → Error → Tree → Correction → Repeat

Part 2 · XGBoost

Extreme Gradient Boosting: gradients, Hessians and regularization

1. What is XGBoost?

XGBoost = Extreme Gradient Boosting: an optimized implementation of gradient-boosted decision trees. Trees are built sequentially; each improves the ensemble by optimizing a loss function.

Widely used for

- House prices & sales
- Fraud detection
- Customer churn
- Disease classification
- Ranking & other tasks

XGBoost = Gradient + Hessian + Regularization + Efficient tree building

2. How Does XGBoost Work?

The first model predicts 60 for everyone: too high for some, too low for others. XGBoost calculates two quantities from the loss:

First-order gradient g

Direction and size of the change in the loss.

Second-order gradient h

Curvature of the loss: how the gradient itself changes.

g and h → useful splits and leaf values, with a penalty on complex trees

XGBoost: Overall Flow



↺ *Gradients → tree → update → repeat*

Prediction → g, h → Best split → Leaf weights → Update → Repeat

4.1 Prediction Equation

$\hat{y}_i$ final prediction for example i

$F_0(x_i)$ initial prediction

$f_k(x_i)$ contribution from tree k

M number of boosting trees

A learning rate η scales each tree: $F_m = F_{m-1} + \eta f_m$. Smaller η = more gradual updates.

$$\hat{y}_i = F_0(x_i) + \Sigma\, f_k(x_i), \quad k = 1 \ldots M$$

4.2 The Objective Function

Loss $l(y_i, \hat{y}_i)$

Measures the prediction error.
Encourages accurate predictions.

Regularization $\Omega(f_k)$

Penalizes the complexity of tree k.
Discourages unnecessarily complex models.

Objective = Loss + Regularization

(N = number of training examples)

$$\text{Obj} = \Sigma_i \, l(y_i, \hat{y}_i) + \Sigma_k \, \Omega(f_k)$$

The Regularization Term Ω(f)

T — number of leaves

w_j — score (weight) of leaf j

γ — penalty for adding leaves

λ — L2 regularization on leaf weights

$$\Omega(f) = \gamma T + \tfrac{1}{2} \lambda \Sigma_j w_j^2, \quad j = 1 \ldots T$$

Why Is Regularization Important?

No penalty

A tree with 20 leaves can fit every training example perfectly.

It may simply memorize the training data.

With Ω(f)

Extra splits and extreme leaf predictions become less attractive.

The model stays simpler and generalizes better.

XGBoost penalizes tree complexity

4.3 First- and Second-Order Gradients

At round t, XGBoost approximates the loss around the current predictions $\hat{y}_i^{(t-1)}$ with a second-order Taylor expansion:

$$L^{(t)} \approx \Sigma_i \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \tfrac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

$$g_i = \partial l(y_i, \hat{y}_i) / \partial \hat{y}_i$$

how the loss changes

$$h_i = \partial^2 l(y_i, \hat{y}_i) / \partial \hat{y}_i^2$$

how the gradient changes

Both derivatives are evaluated at the current prediction

5. XGBoost Numerical Example: Setup

Student	Study hours	Actual marks
A	1	40
B	2	50
C	3	60
D	4	70
E	5	80

Same five students as before

- Initial prediction = mean = 60
- Loss: $l = \frac{1}{2}(\hat{y} - y)^2$
- This choice makes g and h easy to calculate

$$\hat{y}^{(0)} = (40 + 50 + 60 + 70 + 80) / 5 = 60$$

Step 1: Calculate the Gradients

Student	Actual y	Prediction ŷ	Gradient g
A	40	60	+20
B	50	60	+10
C	60	60	0
D	70	60	−10
E	80	60	−20

$g = \hat{y} - y$ · positive: prediction too high · negative: too low

Sign Convention: g vs Residual

Gradient $g = \hat{y} - y$

Positive g: prediction too high → a negative update is desirable.

Negative g: prediction too low → a positive update is desirable.

Residual $= y - \hat{y}$

The convention used in Gradient Boosting earlier.

Opposite sign to g.

Both conventions are valid as long as we use them consistently.

$$g = -(\text{residual})$$

Step 2: Calculate the Hessians

Student	Gradient g	Hessian h
A	+20	1
B	+10	1
C	0	1
D	−10	1
E	−20	1

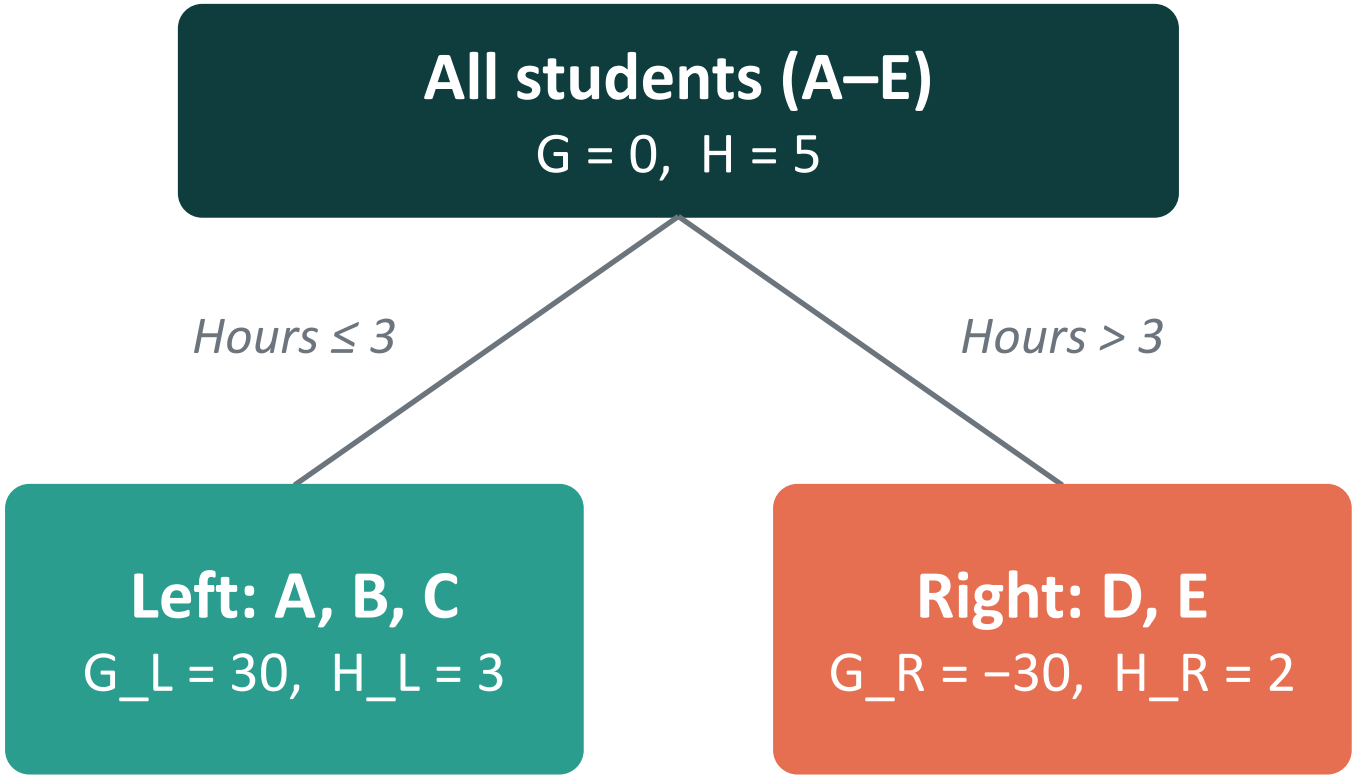
Squared-error loss with ½

$$h_i = \partial^2 l / \partial \hat{y}_i^2 = 1$$

So every training example has a Hessian equal to 1 in this example.

For this loss: $g = \hat{y} - y$ and $h = 1$

Step 3: Propose a Tree Split



Sums in a leaf

$G = \Sigma g_i, \quad H = \Sigma h_i$

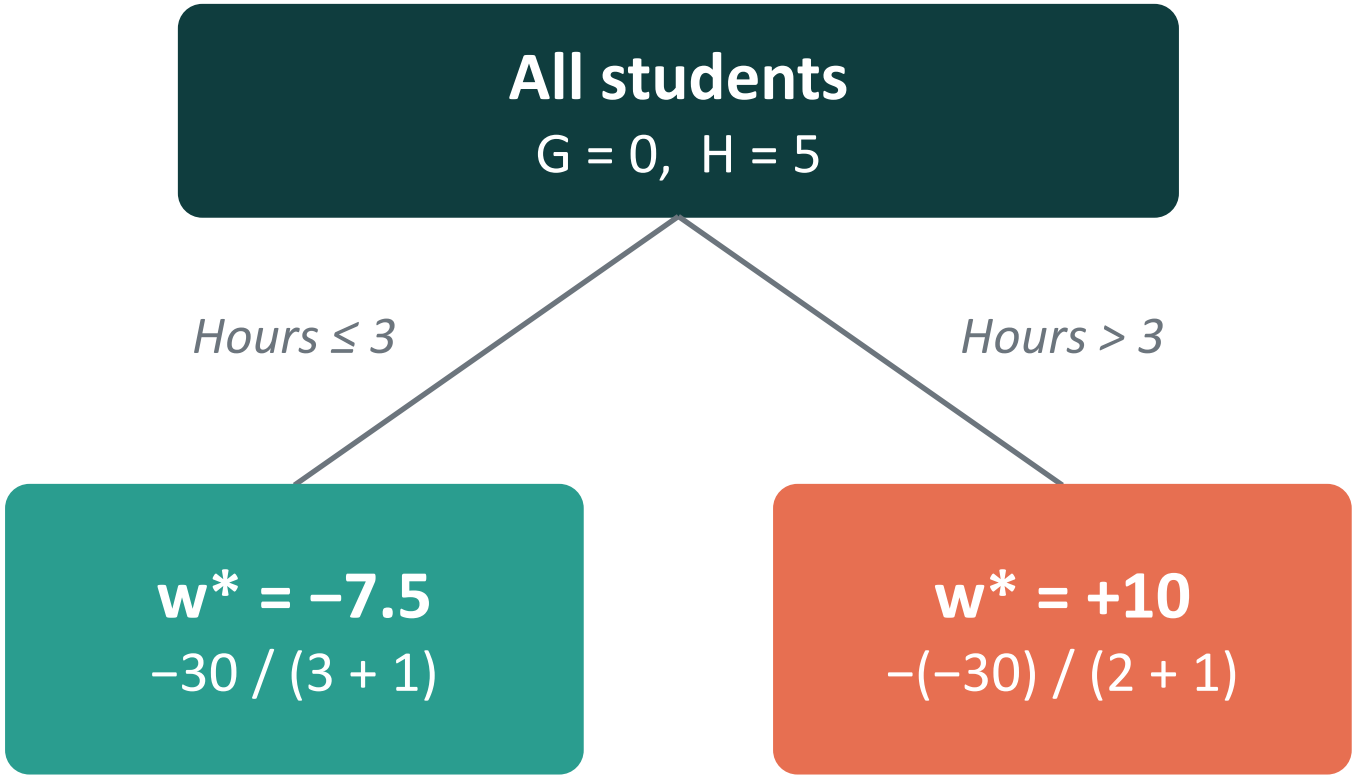
Left: 20 + 10 + 0 = 30; 1 + 1 + 1 = 3

Right: −10 − 20 = −30; 1 + 1 = 2

Parent: G = 0, H = 5

Candidate split: study hours ≤ 3 | study hours > 3

Step 4: Optimal Leaf Weights (λ = 1)



Interpretation

Left leaf: correction −7.5

Right leaf: correction +10

Dividing by H + λ: the Hessian is local curvature, λ regularizes. Larger λ shrinks the update.

$$w_j^* = -G_j / (H_j + \lambda)$$

Step 5: Update the Predictions (η = 0.3)

Student	Actual	$\hat{y}^{(0)}$	Leaf w	$\hat{y}^{(1)}$
A	40	60	−7.5	**57.75**
B	50	60	−7.5	**57.75**
C	60	60	−7.5	**57.75**
D	70	60	+10	**63**
E	80	60	+10	**63**

$$\hat{y}^{(1)} = \hat{y}^{(0)} + \eta\, w^* \quad \text{e.g. } 60 + 0.3(-7.5) = 57.75$$

CLeaf Values

- XGBoost leaf values are not simply the mean residuals
- They come from gradient sums, Hessian sums and λ
- Round 1 is done; XGBoost now recomputes g and h from the new predictions and builds another tree

Same split	GB mean residual	XGBoost w*
Left (hours ≤ 3)	−10	−7.5
Right (hours > 3)	+15	+10

With h = 1: $w^* = \text{mean residual} \times H / (H + \lambda)$ — λ shrinks the update

6. How Does XGBoost Decide the Best Split?

XGBoost does not split just because a feature is strongly related to the target. It evaluates how much a split improves the regularized objective, measured by the split gain.

1

Child quality

Quality of the two child leaves

2

Parent score

Score of keeping the parent leaf

3

γ penalty

Charge for the extra leaf

Gain = improvement in the regularized objective

6.1 Split Gain Formula

G_L, H_L gradient and Hessian sums in the left child

G_R, H_R sums in the right child

G, H parent sums: $G = G_L + G_R$, $H = H_L + H_R$

γ penalty for adding a leaf

$$\text{Gain} = \tfrac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma$$

Child leaves − parent leaf − γ penalty

6.2 Calculate the Gain (λ = 1, γ = 0)

G_L = 30, H_L = 3 G_R = −30, H_R = 2 G = 0, H = 5

225
$30^2 / (3+1)$

\+

300
$(-30)^2 / (2+1)$

−

0
$0^2 / (5+1)$

=

262.5
½ × 525

Gain = ½ [900/4 + 900/3 − 0] = ½ [225 + 300] = 262.5

Gain = 262.5

Effect of γ on the Split

262.5	−	300	=	−37.5
Gain (γ = 0)		γ penalty		Net gain

- Net gain = 262.5 − 300 = −37.5
- Not accepted under the usual positive-gain rule
- The split fails to justify its complexity penalty

$$\text{Gain} - \gamma > 0 \rightarrow \text{keep the split; otherwise reject it}$$

What Should Students Understand?

1. XGBoost evaluates candidate splits mathematically
2. It compares the split’s benefit with the parent leaf’s score
3. It penalizes unnecessary leaves
4. It prefers a split only when the gain justifies the added complexity, subject to tree constraints

Split only when the gain beats the complexity penalty

7. The Complete XGBoost Algorithm

1. Initialize predictions (e.g. the mean target)
2. Calculate gradients g_i and Hessians h_i
3. Build the tree using regularized gain and tree constraints
4. Calculate leaf weights from G, H and λ
5. Update predictions with learning rate η
6. Repeat until M trees or a validation stopping rule

Initialize → g, h → Build tree → Leaf weights → Update → Repeat

3. XGBoost vs Gradient Boosting (1/2)

Feature	Traditional Gradient Boosting	XGBoost
Learning strategy	Sequential boosting	Sequential boosting
Gradient information	Commonly first-order gradients	First- and second-order gradients
Tree complexity	Tree settings and other regularization choices	Explicit regularization of leaf weights and number of leaves in the objective
Split selection	Depends on the implementation	Regularized gain calculation

3. XGBoost vs Gradient Boosting (2/2)

Feature	Traditional Gradient Boosting	XGBoost
Missing values	Depends on implementation	Can learn default directions
Training efficiency	Depends on implementation	Optimized, scalable tree building
Overfitting control	Learning rate, tree depth, other settings	Learning rate, regularization, tree constraints, sampling, early stopping